一、 界面逻辑推导分析

通过观察界面中的红字输入与输出结果，可以总结出以下规律：

1. LTE网络（FDD/TDD）

- **输入条件**：必须先选择**频段（Band）**，然后输入**上行频率**、**下行频率**或**下行频点号**中的任意一个。
- **输出结果**：自动计算出对应的**上行频点号**（如果是输入频率）或**上下行频率**（如果是输入频点）。
- **推导关系**：
  - **B1 (FDD)**：下行 2120 MHz → EARFCN 100。公式：$2110+0.1\times100=2120$ 。
  - **B3 (FDD)**：下行 1870 MHz → EARFCN 1850。公式：$1805+0.1\times(1850-1200)=1870$ 。
  - **B34 (TDD)**：频率 2017.5 MHz → EARFCN 36275。公式：$2010+0.1\times(36275-36200)=2017.5$ 。
  - **结论**：LTE完全遵循 $F=Flow+0.1\times(N-Noffs)$ 的线性关系。

2. NR网络（FDD/TDD）

- **输入条件**：选择频段，输入**下行频率**或**下行频点号 (NARFCN)**，或者输入**GSCN (全局同步信号号)**。
- **输出结果**：计算出对应的频点或频率。
- **推导关系**：
  - **N1 (FDD)**：下行 2114.45 MHz → NARFCN 422890。
    - 验证：N1 属于 0~3GHz 范围，栅格为 5kHz。
    - $2110MHz+0.005MHz\times(422890-422000)=2110+4.45=2114.45$ MHz 。完全吻合。
  - **N41 (TDD)**：频率 2524.95 MHz → NARFCN 504990。
    - 验证：N41 属于 3~24GHz 范围，栅格为 15kHz。
    - $3000MHz+0.015MHz\times(504990-600000)$ ... 等等，这里 N41 的起始频点是 499200 对应 2496MHz。
    - 重新计算：$2496+0.015\times(504990-499200)=2496+86.85=2582.85$ (不对)。
    - 实际上 N41 范围是 499200-537996。2524.95 对应的计算：$(2524.95-3000)/0.015+600000=504990$ 。吻合（使用全局公式）。

二、 Python 换算脚本

以下脚本基于 **3GPP TS 36.101/36.104 (LTE)** 和 **TS 38.101/38.104 (NR)** 协议编写，实现了您截图中软件的核心功能。

1. LTE 频点与频率换算脚本

```
def lte_freq_to_earfcn(band, freq_mhz, direction='DL'):
    """
    LTE 频率转 EARFCN
    :param band: 频段号 (int)
    :param freq_mhz: 频率 (MHz, float)
    :param direction: 'DL' (下行) 或 'UL' (上行)
    """
    # 3GPP TS 36.104 Table 5.7.3-1 (部分常用频段示例)
    lte_bands = {
        1:  {'dl_low': 2110, 'ul_low': 1920, 'dl_off': 0,    'ul_off': 18000},
        3:  {'dl_low': 1805, 'ul_low': 1710, 'dl_off': 1200, 'ul_off': 19200},
        5:  {'dl_low': 869,  'ul_low': 824,  'dl_off': 2400, 'ul_off': 20400},
        8:  {'dl_low': 925,  'ul_low': 880,  'dl_off': 3450, 'ul_off': 21450},
        34: {'dl_low': 2010, 'ul_low': 2010, 'dl_off': 36200,'ul_off': 36200},
# TDD
        38: {'dl_low': 2570, 'ul_low': 2570, 'dl_off': 37750,'ul_off': 37750},
# TDD
        39: {'dl_low': 1880, 'ul_low': 1880, 'dl_off': 38250,'ul_off': 38250},
# TDD
        40: {'dl_low': 2300, 'ul_low': 2300, 'dl_off': 38650,'ul_off': 38650},
# TDD
        41: {'dl_low': 2496, 'ul_low': 2496, 'dl_off': 39650,'ul_off': 39650},
# TDD
    }

    if band not in lte_bands:
        return "该频段未在此脚本中定义，请查阅3GPP TS 36.104"

    b = lte_bands[band]
    f_low = b['dl_low'] if direction == 'DL' else b['ul_low']
    n_off = b['dl_off'] if direction == 'DL' else b['ul_off']

    # 公式: N = N_off + (F - F_low) / 0.1
    earfcn = n_off + (freq_mhz - f_low) / 0.1
    return int(earfcn)

def lte_earfcn_to_freq(band, earfcn, direction='DL'):
    """
    LTE EARFCN 转 频率
    """
    lte_bands = {
```

```
        1:  {'dl_low': 2110, 'ul_low': 1920, 'dl_off': 0,    'ul_off': 18000},
        3:  {'dl_low': 1805, 'ul_low': 1710, 'dl_off': 1200, 'ul_off': 19200},
        5:  {'dl_low': 869,  'ul_low': 824,  'dl_off': 2400, 'ul_off': 20400},
        8:  {'dl_low': 925,  'ul_low': 880,  'dl_off': 3450, 'ul_off': 21450},
        34: {'dl_low': 2010, 'ul_low': 2010, 'dl_off': 36200,'ul_off': 36200},
        38: {'dl_low': 2570, 'ul_low': 2570, 'dl_off': 37750,'ul_off': 37750},
        39: {'dl_low': 1880, 'ul_low': 1880, 'dl_off': 38250,'ul_off': 38250},
        40: {'dl_low': 2300, 'ul_low': 2300, 'dl_off': 38650,'ul_off': 38650},
        41: {'dl_low': 2496, 'ul_low': 2496, 'dl_off': 39650,'ul_off': 39650},
    }

    if band not in lte_bands:
        return "该频段未在此脚本中定义"

    b = lte_bands[band]
    f_low = b['dl_low'] if direction == 'DL' else b['ul_low']
    n_off = b['dl_off'] if direction == 'DL' else b['ul_off']

    # 公式: F = F_low + 0.1 * (N - N_off)
    freq = f_low + 0.1 * (earfcn - n_off)
    return freq

# --- 测试案例 (对应截图) ---
print("--- LTE 测试 ---")
print(f"B1 下行 2120MHz -> EARFCN: {lte_freq_to_earfcn(1, 2120, 'DL')}") # 应为
100
print(f"B3 下行 1870MHz -> EARFCN: {lte_freq_to_earfcn(3, 1870, 'DL')}") # 应为
1850
print(f"B34 下行 2017.5MHz -> EARFCN: {lte_freq_to_earfcn(34, 2017.5, 'DL')}")
# 应为 36275
print(f"B1 EARFCN 100 -> 下行频率: {lte_earfcn_to_freq(1, 100, 'DL')} MHz")
```

2. NR (5G) 频点与频率换算脚本

NR 的计算不依赖具体的 Band 起始点，而是依赖**全局频率范围**（0-3GHz, 3-24GHz, 24-100GHz）。

```
def nr_freq_to_arfcn(freq_mhz):
    """
    NR 频率转 NR-ARFCN
    :param freq_mhz: 频率 (MHz)
    """
    if 0 <= freq_mhz <= 3000:
        # 0-3000 MHz, 栅格 5kHz
        return int(freq_mhz * 1000 / 5)  # 即 freq_mhz * 200
```

```
    elif 3000 < freq_mhz <= 24250:
        # 3000-24250 MHz, 栅格 15kHz
        return int(600000 + (freq_mhz - 3000) * 1000 / 15)
    elif 24250 < freq_mhz <= 100000:
        # 24250-100000 MHz, 栅格 60kHz
        return int(2016667 + (freq_mhz - 24250.08) * 1000 / 60)
    else:
        return "频率超出 NR 范围"

def nr_arfcn_to_freq(arfcn):
    """
    NR-ARFCN 转 频率
    :param arfcn: NR-ARFCN (int)
    """
    if 0 <= arfcn <= 599999:
        # 0-3000 MHz
        return arfcn * 5 / 1000  # 单位 MHz
    elif 600000 <= arfcn <= 2016666:
        # 3000-24250 MHz
        return 3000 + (arfcn - 600000) * 15 / 1000
    elif 2016667 <= arfcn <= 3279165:
        # 24250-100000 MHz
        return 24250.08 + (arfcn - 2016667) * 60 / 1000
    else:
        return "ARFCN 超出 NR 范围"

# --- 测试案例 (对应截图) ---
print("\n--- NR 测试 ---")
# N1: 2114.45 MHz
print(f"NR 2114.45 MHz -> ARFCN: {nr_freq_to_arfcn(2114.45)}") # 应为 422890

# N41: 2524.95 MHz
print(f"NR 2524.95 MHz -> ARFCN: {nr_freq_to_arfcn(2524.95)}") # 应为 504990

# 反向验证
print(f"NR ARFCN 422890 -> 频率: {nr_arfcn_to_freq(422890)} MHz")
print(f"NR ARFCN 504990 -> 频率: {nr_arfcn_to_freq(504990)} MHz")
```

三、 脚本使用说明

1. **输入参数：**
   - **LTE：**需要 `band` (频段号), `freq_mhz` (频率值) 或 `earfcn` (频点值), 以及 `direction` ('DL'下行 或 'UL'上行)。
   - **NR：**只需要 `freq_mhz` 或 `arfcn`，因为 NR 使用全局公式，不需要指定 Band 即可计算（但在实际工程中，算出 ARFCN 后需校验是否在对应 Band 的范围内）。

2. **输出结果：**
   - 脚本会输出对应的频点号（整数）或频率值（MHz，浮点数）。
3. **GSCN 换算：**
   - 截图中还出现了 **GSCN (全局同步信道号)**。GSCN 的计算比 ARFCN 复杂，它依赖于具体的 SSB 子载波间隔。如果您需要 GSCN 的换算逻辑，可以进一步告知，我可以补充相关代码。